STAC AI BOOTCAMP NYC



Hands-on adaptation: from hypothesis to evidence

Matthew Dixon, PhD, FRM
BeliefLens

21 October 2026 | 1:15–2:15

matthew.dixon@belieflens.org

Start with the behaviour you want to change

Notebook: `04_adaptation_lab.ipynb`

Name a checkable failure.

"Make the model better" does not tell us what to train or how to judge success. Choose a recurring behaviour that a reviewer can recognize.

Concrete example

A request was waiting yesterday and approved today. The model keeps reporting the old status.

A useful target is narrow enough to check but broad enough to recur. We can then ask whether the same change helps on new examples.

`https://colab.research.google.com/github/mfrdixon/STAC_AI_Bootcamp/blob/main/notebooks/04_adaptation_lab.ipynb`

A training run produces a candidate

Decide whether to use it.

We compare the candidate with the unchanged model on the same cases. Count both errors it fixes and errors it introduces.

Concrete example

Correctly reading the new approval is useful only if the change does not also invent approvals when no update exists.

A changed answer is not automatically a better answer. The comparison must tell us which requirements are now met and which have become less reliable.

Our route: six decisions in order

Decision	Why we need it
Diagnose	Establish which component failed.
Specify	Write the input and the desired response.
Protect	Reserve cases for selection and final assessment.
Configure	Choose the permitted update and resources.
Compare	Inspect matched outputs and regressions.
Qualify	Accept, revise or reject for a defined scope.

Each decision supplies what the next one needs. Choosing a training setting becomes meaningful only after we know the behaviour and comparison it must support.

What you will be able to explain

Ability	What you should be able to point to
Define the learning target	An input, a reviewed response and supervised tokens.
Explain the update	Frozen weights and trainable adapter matrices.
Protect the comparison	Distinct training, validation and final-test roles.
Compare behaviour	Corrections and regressions on identical cases.
Choose a repair	A failed handoff and its tested intervention.

The goal is to explain an experiment to another reviewer. Point to the input, update and observed outcomes rather than relying on the name of the training method.

First check whether the information arrived

Missing evidence calls for retrieval.

If the latest update never reached the model, there is no supplied update for it to interpret. Inspect the actual context before changing weights.

Concrete example

The document collection contains today's approval, but the prompt includes only yesterday's waiting status.

First confirm that the required information reaches the model. Then test whether it interprets that information correctly; this separates retrieval from interpretation.

Then locate the remaining failure

Interpretation and handoff errors need different tests.

Repeatedly selecting the old status from correct context can justify adaptation. Correct extraction followed by a corrupted tool argument calls for application repair.

Concrete example

Model output: approved. Database request: waiting. Repair the code that builds the request.

Several faults can produce the same final answer. Recording intermediate values helps us distinguish a model interpretation problem from a later software defect.

Write the input and target before training

Make the contract explicit.

Define what information the model receives, what response is acceptable and how missing or contradictory information should be represented.

Concrete example

Input: old status + latest update. Target: current status + supporting words. If no current update exists, return unknown.

The target should say what to do when support is absent, not only when the answer is obvious. Otherwise demonstrations may teach a confident response to incomplete information.

Define success before inspecting outcomes

Keep the baseline and assessment fixed.

Use the unchanged model with the same prompt and evidence. Evaluate unseen records and inspect difficult subsets, including missing updates and contradictory records.

Concrete example

A candidate that improves easy approvals but invents approval on missing-update cases may fail the intended task.

A fixed comparison prevents us from changing the definition of success to suit the result. Difficult cases reveal whether an apparent improvement is dependable.

One labelled example: what is input, what is target?

Role	Illustrative record
System instruction	Extract the latest supported status; otherwise return unknown.
User input	Old status: waiting. Latest update: the request was approved.
Reviewed assistant target	Status: approved. Evidence: the request was approved.

During training the reviewed assistant response supplies the tokens to learn. During use, only the instruction and evidence are supplied; the model must produce the response itself.

The target teaches the behaviour we reward

A wrong target teaches the wrong answer.

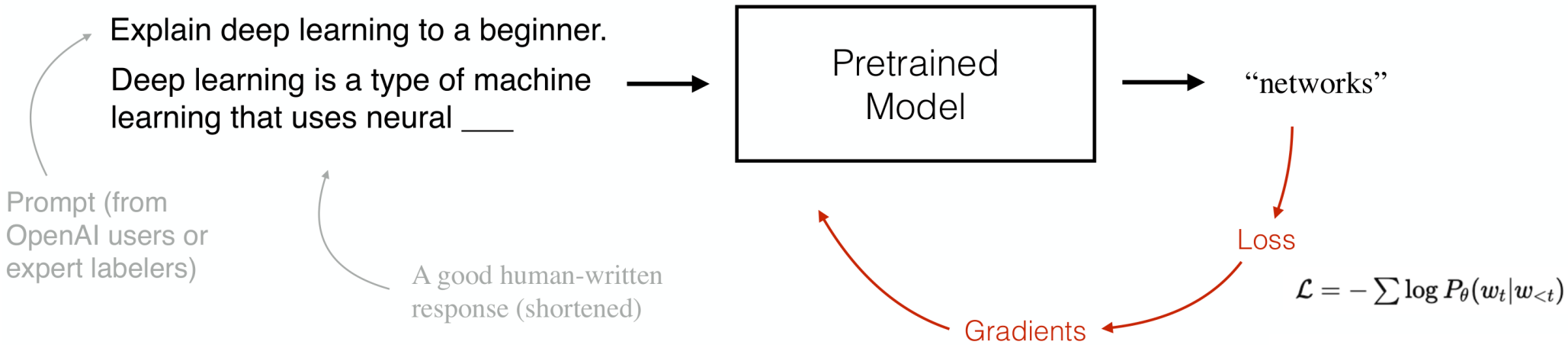
Training increases the probability of reviewed response tokens. Including a supporting quotation teaches a different output contract from training on a status word alone.

Concrete example

Input: old status waiting; no current update. ewline Reviewed target: Status: unknown. Evidence: no current update supplied.

Reviewing labels is part of engineering the behaviour. A consistent but incorrect target can be learned successfully while making the application less useful.

How a reviewed response changes the model



The model predicts a target token. A loss penalizes low probability for that token. Gradients guide a small change to trainable parameters.

The gradient describes how a small parameter change affects the prediction loss. Reducing that loss is a learning step; whether the resulting answers help still requires evaluation.

The loss scores the response tokens we choose

$$\mathcal{L}(\theta) = -\frac{1}{\sum_t m_t} \sum_t m_t \log p_\theta(y_t \mid x, y_{<t})$$

- θ: parameters; $\mathcal{L}$: mean loss; x: prompt; t: response token index; y_t: target token; $y_{<t}$: earlier target tokens.
- p_θ: model probability; m_t: 1 for scored tokens, 0 otherwise; Σ: sum; log: natural logarithm; $|$: conditional on.
- Read: average negative log probability over selected demonstrated tokens. Require at least one scored token; loss is nats per token.
- In this lab inspect the stored masking/training specification; low loss alone cannot qualify the full workflow.

Teacher forcing supplies the correct earlier words

$$p_\theta(y \mid x) = \prod_{t=1}^{T} p_\theta(y_t \mid x, y_{<t})$$

- x: prompt; y: full demonstrated response; T: response length; t: token position.
- y_t: current target token; $y_{<t}$: preceding demonstrated tokens; θ: model parameters; p_θ: probability.
- $\prod$: multiply across positions; $\mid$: conditional on. Probability is unitless.
- Read: sequence probability is the product of next-token probabilities. During training earlier target tokens are supplied; during generation earlier model outputs are used.

The loss mask does not hide the input

Positions	Visible as context?	Contribute to loss?
Instruction + user record	Yes	No: mask 0
Reviewed response + chosen end token	Yes	Yes: mask 1

These are illustrative positions, not actual tokenizer IDs. Input words still help predict the response even when their direct loss is zero.

A zero mask removes a position from the direct penalty, not from the conversation. The evidence can therefore guide the answer without being a text passage the model is trained to repeat.

How examples are weighted changes what is learned

Specify the averaging rule.

Averaging all supervised tokens gives long responses more influence. Averaging each example's mean gives examples equal influence. Repeating a record increases its weight.

Concrete example

Oversampling a rare class can balance training, but it does not create new independent information.

Weighting defines which mistakes matter most to the optimizer. Check that the chosen balance matches the task before interpreting a lower average loss.

Inspect the processed training record

The model sees tokens, not your original file.

Check role separators, the response positions selected for loss and the sequence after truncation. A correct source record can become an incomplete model input.

Concrete example

A long request history ends with “approved.” A length limit removes that final update. The remaining input no longer supports approval.

Inspect one ordinary record and one long or incomplete record after preprocessing. These small checks can reveal a systematic problem before it is repeated across the whole dataset.

Give each dataset one job

Fit, select, then assess.

Training adjusts parameters. Validation chooses settings or a checkpoint. The final test assesses the selected candidate after those choices are frozen.

Concrete example

Practice questions teach; a mock exam guides selection; a final exam checks the chosen preparation.

The separation is about how information is used, not which folder contains it. Once a result influences a model choice, it is part of development.

Separate rows may still share information

Match the split to the claim.

Copies, revised documents, repeated entities and overlapping outcome windows can cross a row split. A later period with familiar entities does not test new-entity generalization.

Concrete example

Training on yesterday's version of a record and testing on a near-identical revision may reward familiarity.

The split should support the specific claim we intend to make. Success on later records from familiar companies does not establish success on unfamiliar companies.

Question: which result should select the checkpoint?

Choice	Selection rule
A	Inspect final-test performance after every epoch.
B	Use the validation metric defined before the run.
C	Choose the candidate with the most impressive example.

A checkpoint is a saved model state from a stage of training. Consider how each choice uses information, then commit to an answer before continuing.

Answer: B, use validation for selection

Keep final assessment separate.

Validation supports choosing a candidate. Training loss diagnoses fit. The untouched final test assesses that chosen procedure, including its selection decisions.

Concrete example

Choose epoch two using validation, freeze it, then assess it on the final records.

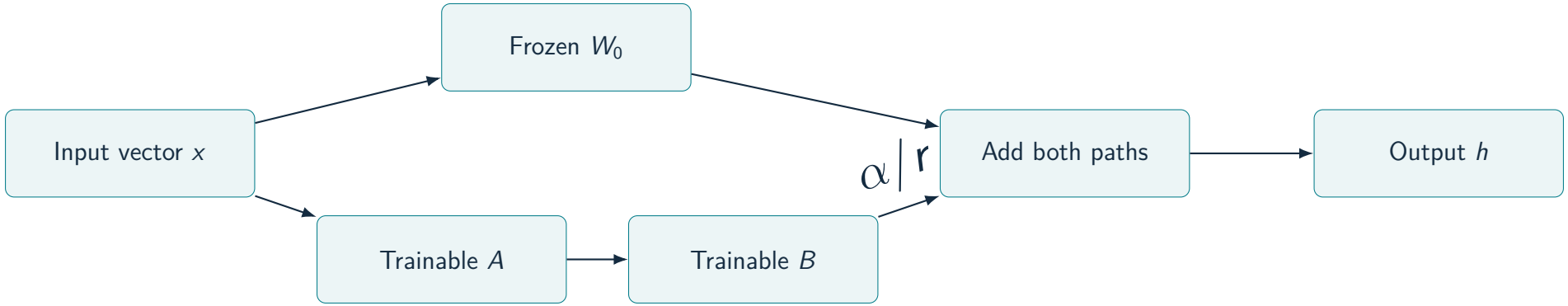
The selected candidate still has to pass the final assessment. If that assessment leads to another change, a new independent check is needed for the revised candidate.

LoRA learns a correction to a frozen transformation

$$h = W_0 x + \frac{\alpha}{r} BAx$$

- x: input vector; h: output vector; W_0: frozen base matrix, with output-by-input shape.
- A: trainable input-to-rank matrix; B: trainable rank-to-output matrix; r: positive integer rank; α: scale; α/r: unitless multiplier.
- Read: add a learned low-rank correction to the same base transformation. Vector units depend on the representation; weights map between them.
- Only the selected adapter matrices are updated. This is a parallel branch inside a layer, not a smaller replacement model or a layer inserted between every pair of layers.

The same input follows two paths



The upper path keeps the original transformation. The lower path learns a correction. Their outputs are added inside the model.

The correction is added inside a selected transformation, so it can change what later layers receive. It is not a separate smaller model replacing the original network.

x: input; h: output; W_0: frozen matrix; A, B: trainable correction matrices; r: positive rank; α: scale. The multiplier α/r is unitless.

Count the parameters that can change

$$A \in \mathbb{R}^{r \times d_{\text{in}}}, \quad B \in \mathbb{R}^{d_{\text{out}} \times r}, \qquad N_{\text{adapter}} = r(d_{\text{in}} + d_{\text{out}})$$

- A, B: trainable matrices; r: rank; $d_{\text{in}}, d_{\text{out}}$: input/output widths; all dimensions are positive integers.
- $\mathbb{R}$: real-valued entries; $\in$: belongs to; $\times$: matrix row-by-column shape; N_{adapter}: number of trainable entries.
- Read: count entries in A plus entries in B. Counts are unitless; exclude biases and any other trainable modules.
- For 1024-by-1024 projection and rank 8: 16,384 adapter entries versus 1,048,576 base entries. Larger rank is a capacity choice, not guaranteed improvement.

LoRA makes an assumption we must test

A restricted update may be enough.

The product BA has rank at most r. We assume a useful task change can be approximated within that restricted family and in the selected transformations.

Concrete example

A compact correction may fix current-status extraction. It might also lack the capacity or placement needed for the task.

Low rank limits what the update can express. Whether that restriction is adequate is an empirical question for the task, not a guarantee supplied by the method.

BA: matrix product of trainable factors B and A; r: maximum correction rank, a positive integer.

Choose which transformations may change

Treat target modules as an experiment.

Query/value projections are one possible configuration. Adding key or other transformations changes the family of permitted updates and often the parameter budget.

Concrete example

Compare query/value against query/key/value using the same task and assessment, with a declared compute or parameter budget.

Changing the allowed transformations changes the experiment. Keep the assessment fixed so any benefit or regression can be compared under a clearly stated budget.

Choose the size and duration of the update

Settings control different things.

Rank controls update dimension; alpha its scale. Learning rate and steps control optimization. Sequence length and microbatch affect context and memory. Validation schedule controls selection evidence.

Concrete example

A longer sequence may retain the decisive exception, while a larger rank does not restore an exception that preprocessing removed.

Settings interact, so record the configuration instead of attributing a result to one knob afterward. Inspect actual trainable parameters and the context that reaches the model.

Memory-saving methods solve resource problems

Each saves something different.

QLoRA keeps a quantized frozen base and trains adapters. Gradient accumulation combines smaller batches before updating. Checkpointing recomputes activations to save their memory.

Concrete example

Accumulating eight small batches can replace one larger update batch. It cannot make a single oversized record fit.

First verify that a short run can complete an update, save and reload. Resource feasibility lets the experiment run; it does not show that the learned behaviour is better.

What one training step actually does

Operation	Meaning
`model(**batch).loss`	Predict target tokens and calculate their penalty.
`loss.backward()`	Calculate gradients for the permitted update.
`optimizer.step()`	Use gradients to change trainable parameters.
`optimizer.zero_grad()`	Clear gradients at the appropriate update boundary.

Only parameters permitted to train should be updated. Validation computes predictions and metrics without performing this optimizer step.

Saving and reloading is part of the experiment

A metric must identify a reproducible candidate.

Save the adapter, compatible base revision, tokenizer, template, configuration and evaluation outputs. Check that the saved candidate reloads before relying on it.

Concrete example

Two adapters with the same file name but different base revisions are not necessarily interchangeable.

A result is useful only if we know which saved state produced it. Preserve the associated data, template and configuration so another reviewer can repeat the comparison.

Choose the route that answers your question

Route	What actually happens
Recorded adaptation	Inspect real saved training and paired outputs; no key or GPU.
Controlled workflow repair	Change a supplied faulty value and recompute dependencies.
Optional instructor training	Perform fresh parameter updates on prepared hardware.

The default activities run without a provider key. Recorded results support inspection, while the controlled repair lets you change a value and observe its consequences.

Exercise A: inspect what was trained

Open the readable adaptation notebook.

Run sections 1–2 of `04_actual_adaptation_readable.ipynb`. Find the input, target, trainable modules and selected epoch.

Concrete example

Be able to finish: "The model received [input] and was trained to output [target]."

Look for the exact target rather than assuming it from the task name. Confirm whether the model learned a class token, a prose answer or another response format.

https://colab.research.google.com/github/mfrdixon/STAC_AI_Bootcamp/blob/main/notebooks/04_actual_adaptation_readable.ipynb

Exercise A: compare one matched case

Change the case, then reveal the outcome.

In section 3 change `CASE_INDEX`. Predict before revealing the reference outcome. Run sections 4–6 to inspect checks, aggregate results and split dates.

Concrete example

State whether the adapter's changed answer corrected an error or introduced one.

One interesting case explains a change; the full set assesses its pattern. Keep corrections and regressions in the comparison instead of choosing only favourable examples.

`https://colab.research.google.com/github/mfrdixon/STAC_AI_Bootcamp/blob/main/notebooks/04_adaptation_lab.ipynb`

The recorded run: what these results refer to

Recorded item	Value
Base	Qwen2.5-0.5B-Instruct; 494,032,768 base parameters.
Update	Query/value LoRA; rank 8; alpha 16.
Training data	527 distinct windows; 777 balanced records per epoch.
Resources	One T4; recorded duration 28.6 minutes.
Selection	Epoch 2 by validation loss.

The balanced records include repeated influence from existing windows. The model size and runtime identify this run; they are not a promise about another dataset or environment.

A real labelled record, condensed for readability

Field	Verified saved-record excerpt
Input cutoff	CVX; 9 October 2025, 06:30 UTC.
Price input	21 daily closes through 8 October; latest USD 153.7400; prior 20-session return -2.5667%.
Other input	News, financial facts and SPY-relative context.
Target output	`C`: realized five-session excess return above +1%.

The future return becomes a target only after the outcome is observed. It must not be supplied as part of the earlier forecasting evidence.

The financial target is not a truth certificate

The model learns a class-token mapping.

The target does not certify evidence quality, predict a profitable authorized trade or provide calibrated confidence. Inspect the exact historical prompt contract.

Concrete example

The saved system requests A/B/C, while its nested question requests named signals and abstention. That mismatch is an audit finding.

A timestamp check can pass even when the news is irrelevant to the company. Evidence eligibility and target prediction are different requirements and should be inspected separately.

Why epoch two was selected

Loss	Epoch 1	Epoch 2	Epoch 3
Mean logged training	0.3783	0.3563	0.3510
Validation	0.4040	**0.3692**	0.3770

Training fit keeps improving. The separate validation metric is lowest at epoch two. That identifies the candidate to assess next.

Validation selects which saved state to assess. The test of complete predictions still follows; a lower token loss alone does not establish a better financial decision.

Count corrections and regressions together

Same 16 cases	Base	Adapter
Correct labels	6/16	3/16
Base errors corrected	—	3
Previously correct cases broken	—	6

The adapter changes behaviour, but this comparison does not establish improved overall label accuracy.

Six previously correct cases were lost and three previously wrong cases were fixed. That paired account shows why changed behaviour alone is not an improvement claim.

Why a different metric can appear to improve

Balanced accuracy gives classes equal weight.

It rises from 33.3% to 40.7%, while ordinary accuracy falls. Here there are six negative outcomes, one neutral and nine positive. The single neutral case has large influence.

Concrete example

Correctly recognizing that one neutral outcome contributes one third of the class-averaged metric.

Metric choice determines how errors are weighted. Report the class counts and regressions so a single rare outcome cannot make the result look more convincing than it is.

The final answer does not locate the failure

Inspect the handoffs. Follow evidence → extraction → tool arguments → calculation → allowed action. Check the recorded input and output at each step.

Concrete example

The source and extraction say approved, but the application's request says waiting. That mismatch is observable.

The graph records observable relationships between artifacts. It can locate a failed check without claiming to reveal the model's hidden internal reasoning.

Use the graph to target a repair

Recheck what depends on the changed value.

If an input is repaired, reassess its dependent calculations and claims. Keep independent permissions and other faults visible.

Concrete example

A corrected extraction can repair a report while a separate wrong calculator argument still blocks the action.

A changed input may affect several dependent outputs, but it need not change all of them. Verify the actual recomputation and retain any separate unresolved requirement.

Exercise B: inspect the actual calculator request

Compare source values with tool arguments.

Open `04_workflow_diagnosis.ipynb`. Set `CASE_ID='arguments'` in C02. The source says consensus 1.00, but the request passes 2.00.

Concrete example

Actual earnings: 1.20. Correct expected earnings: 1.00. The input discrepancy appears before the calculator runs.

The calculator can only use the values it receives. Compare those values with the evidence before blaming its formula or retraining the language model.

https://colab.research.google.com/github/mfrdixon/STAC_AI_Bootcamp/blob/main/notebooks/04_workflow_diagnosis.ipynb

Correct arithmetic can use the wrong input

$$\text{surprise} = \frac{\text{actual} - \text{consensus}}{|\text{consensus}|}$$

Inputs sent to calculator	Result
Correct: actual 1.20; consensus 1.00	$(1.20 - 1.00)/1.00 = +20\%$
Faulty: actual 1.20; consensus 2.00	$(1.20 - 2.00)/2.00 = -40\%$

Both calculations are arithmetically correct. Only one uses the source-supported consensus.

The inputs must share company, period, units and basis, and consensus must be nonzero. Correcting arithmetic cannot compensate for using an unsupported input.

Actual and consensus are earnings per share in the same currency and period. Bars mean absolute value; require nonzero consensus. Surprise is unitless, shown as a percentage.

Exercise B: repair one fault, then inspect what remains

Predict before applying the repair.

Set `REPAIR_ID='tool_request'` in C04. Then try `two_faults`: repair extraction alone and inspect the remaining argument defect.

Concrete example

A repaired figure can restore one report while the trading path remains withheld.

A successful local repair is not permission to mark the whole workflow correct. The second case demonstrates why remaining checks and independent authority still matter.

`https://colab.research.google.com/github/mfrdixon/STAC_AI_Bootcamp/blob/main/notebooks/04_adaptation_lab.ipynb`

Question: which intervention addresses this defect?

The model extracts the correct value. Application code changes it before calling the calculator.

Choice	Next step
A	Train on more examples of the desired final action.
B	Repair the argument handoff and rerun its checks.
C	Increase adapter rank until the action changes.

Use the recorded mismatch to choose an intervention. Before continuing, state which observation supports your choice and what you would expect to change afterward.

Answer: B, repair the argument handoff

Verify the input-to-source relationship.

Correct the application request, then recompute affected outputs. Retain unrelated faults and independent permission checks.

Concrete example

An approved request may now be recorded correctly, while the operator still lacks authority to release it.

The repaired request should agree with its supporting evidence. A newly correct final answer would not by itself prove that the handoff was repaired.

Decide using the full experimental evidence

Accept, revise or reject for a defined scope.

Review task performance, uncertainty, regression cases, evidence checks and execution permissions. A falling loss curve is only one part of that record.

Concrete example

A model that improves ordinary cases but invents unsupported approvals may need rejection or a narrower permitted use.

A narrow, justified use may be acceptable even when a broad claim is not. State the remaining uncertainty and the consequences of the errors that remain.

Make the decision reproducible and accountable

Keep the artifacts and name the owner.

Record data and split versions, model and adapter revisions, configuration, outputs and limitations. Specify permitted scope, monitoring and rollback.

Concrete example

Another reviewer should be able to identify the selected candidate and explain why it was accepted or rejected.

Reproducible evidence makes a decision inspectable, but it cannot cure a biased assessment. The owner must also specify when new evidence triggers review or rollback.

Explain your conclusion in three sentences

1. What observable behaviour or handoff failed?
2. Which intervention would you test, and why?
3. What evidence would make you accept or reject it?

Use the input, target, permitted update, matched results and workflow checks to support your answer.

A useful conclusion may be that no model training is needed. The important result is a justified intervention and a clear test of whether it worked.